

第八次编程实践课

软件工程与计算 I · 2026 · 南京大学

| Week 8 · 06-Control 答案解析 · 07-TCO 尾递归优化布置

今日安排

知识讲解

大作业 06-Control 答案深度解析

handleDo · handleIf · handleFnStar · cond · when ·
deepClone

大作业 07-TCO 尾递归优化布置

递归 vs 迭代 · 尾位置 · while 循环 · LFunctionTco · def /
defn

核心转变

思维升级 从递归到迭代

- Project 05: 引入环境
- Project 06: 引入控制流 (闭包)
- **Project 07: 解决栈溢出** ← 本次课
- Project 08: eval / atom 高级特性

目标: 理解"为什么递归会栈溢出", 学会用 TCO 把递归
转为迭代。

PART 1 · 06-Control 答案深度解析

handleDo · handleIf · handleFnStar · 扩展 cond/when

大作业进度回顾

解释器流水线

用户输入字符串

- ↓ 【01-REPL】 交互式读取 ← 已完成 ✓
- ↓ 【02-Lexer】 词法分析 ← 已完成 ✓
- ↓ 【03-AST】 语法分析 ← 已完成 ✓
- ↓ 【04-Eval】 求值 ← 已完成 ✓
- ↓ 【05-Env】 环境与绑定 ← 已完成 ✓
- ↓ 【06-Control】 控制流 (do/if/fn*) ← 答案解析 ★
- ↓ 【07-TCO】 尾递归优化 ← 本次布置 ★
- ↓

输出结果

06-Control 答案 — 整体架构

Evaluator.java 新增的符号与 handlers

Evaluator.java

— SYMBOL_DO, SYMBOL_IF, SYMBOL_FN_STAR	← 三个核心控制构造
— SYMBOL_COND, SYMBOL_WHEN, SYMBOL_ELSE	← 两个扩展
— handleDo(list, env)	← 顺序执行
— handleIF(list, env)	← 条件分支
— handleFnStar(list, env)	← 函数闭包（最核心）
— handleCond(list, env)	← 扩展1：多分支
— handleWhen(list, env)	← 扩展2：条件块
— eval 分发：新增 5 个 branch	

关键变化： `handleFnStar` 返回 `LFunction` 并用 `env.deepClone()` 捕获环境快照——这是理解闭包的关键。

06 答案 — handleDo 顺序执行

代码

```
private static LObject handleDo(LList list, Environment env) {
    LObject result = new LList();
    for (int i = 1; i < list.content.size(); i++) {
        result = eval(list.content.get(i), env);
    }
    return result;
}
```

执行过程

```
(do (* 1 2) (- 1 2) (+ 1 2))
i=1: eval(* 1 2) → 2    (丢弃)
i=2: eval(- 1 2) → -1   (丢弃)
i=3: eval(+ 1 2) → 3    (保留)
返回 3
```

要点

要点	说明
从 i=1 开始	跳过 <code>do</code> 本身
每轮都 eval	前面的结果被丢弃
保留最后一个	作为返回值
空 <code>(do)</code>	返回 <code>LList()</code>

和 Project 05 的 begin 关系

完全相同——只是名字换了：`begin` → `do`。

设计模式：顺序执行是最简单的特殊形式，本质就是一个丢弃中间结果的 for 循环。

06 答案 — handleIF 条件求值

```
private static LObject handleIF(LList list, Environment env) {
    final LObject condition = eval(list.content.get(1), env);
    if (condition.equals(new LList()) || condition.equals(new LInteger(0))) {
        return eval(list.content.get(3), env);    // else 分支
    } else {
        return eval(list.content.get(2), env);    // then 分支
    }
}
```

"假"的定义 (Project 06)

值	真假	备注
<code>nil</code> / <code>()</code>	假	空列表
<code>0</code>	假	⚠️ 与 Project 05 不同!
其他整数 / 字符串 / 非空列表	真	

注意: `if` 只求值一个分支——这就是"特殊形式" vs "普通函数调用"的根本区别。如果像普通函数那样先求值所有参数, `(if 1 result (error))` 会触发错误。

06 答案 — handleFnStar 闭包（最核心）

```
private static LFunction handleFnStar(LList list, Environment env) {
    final LList params = (LList) list.content.get(1);           // 参数列表
    final LObject body = list.content.get(2);                   // 函数体
    Environment captured = env.deepClone();                     // ★ 深拷贝环境
    return new LFunction(params.content.size(),
        args -> eval(body, new Environment(captured, params, args)));
}
```

三个关键点

步骤	代码	作用
取参数表	list.content.get(1)	如 (a b)
取函数体	list.content.get(2)	不求值，留到调用时
深拷贝环境	env.deepClone()	★ 闭包的本质

06 答案 — 为什么必须 deepClone?

场景

```
(let* ((x 10)
      (f (fn* (y) (+ x y)))) ; 捕获 x=10
      (x 99))                ; 后续改 x
(f 5))
```

正确结果 = 15 (闭包中的 x 永远是 10)

浅拷贝会出错

1. $env0 = \{x=10\}$
2. f 浅拷贝 $env0 \rightarrow$ 共享符号表
3. 绑定 $x=99 \rightarrow env0 = \{x=99\}$
★ f 里的 x 也变 99!
4. $(f\ 5) \rightarrow x+y = 99+5 = 104$ ✗

deepClone 的正确行为

1. $env0 = \{x=10\}$
2. f 深拷贝 $env0 \rightarrow$ 独立副本 $\{x=10\}$
3. 绑定 $x=99 \rightarrow env0 = \{x=99\}$
★ f 的副本仍是 $\{x=10\}$
4. $(f\ 5) \rightarrow x+y = 10+5 = 15$ ✓

函数式的核心承诺

闭包 = 函数 + 创建时的环境快照

这个"快照"必须是不可变的——否则闭包行为会随外部状态改变而改变，破坏引用透明性。

06 答案 — 扩展1 handleCond

完整代码

```
private static LObject handleCond(
    LList list, Environment env) {
    for (int i = 1; i < list.content.size(); i++) {
        LList clause = (LList) list.content.get(i);
        LObject test = clause.content.get(0);

        if (SYMBOL_ELSE.equals(test)) {
            return eval(clause.content.get(1), env);
        }

        LObject result = eval(test, env);
        if (!result.equals(new LList())
            && !result.equals(new LInteger(0))) {
            return eval(clause.content.get(1), env);
        }
    }
    return new LList(); // 无匹配
}
```

执行过程 (cond (0 10) (1 20) (else 30))

```
i=1: clause=(0 10)
      test=0 → eval=0 → 假 → 跳过
i=2: clause=(1 20)
      test=1 → eval=1 → 真 → 返回 20
```

设计要点

要点	说明
SYMBOL_ELSE 静态比较	不 eval else 符号
首个真值即返回	短路求值
遍历完无匹配	返回 LList()

警惕：else 是符号，不是关键字——需要预先定义常量比较。

06 答案 — 扩展2 handleWhen + eval 分发

handleWhen

```
private static LObject handleWhen(
    LList list, Environment env) {
    LObject cond = eval(list.content.get(1), env);
    if (cond.equals(new LList())
        || cond.equals(new LInteger(0))) {
        return new LList(); // 假 → 返回空
    }
    LObject result = new LList();
    for (int i = 2; i < list.content.size(); i++) {
        result = eval(list.content.get(i), env);
    }
    return result; // 真 → 执行后续并返回最后一个
}
```

eval 核心分发

```
if (SYMBOL_LET.equals(head))
    return handleLet(list, env, false);
else if (SYMBOL_LET_STAR.equals(head))
    return handleLet(list, env, true);
else if (SYMBOL_DO.equals(head))
    return handleDo(list, env);
else if (SYMBOL_IF.equals(head))
    return handleIF(list, env);
else if (SYMBOL_FN_STAR.equals(head))
    return handleFnStar(list, env);
else if (SYMBOL_COND.equals(head))
    return handleCond(list, env);
else if (SYMBOL_WHEN.equals(head))
    return handleWhen(list, env);
else { /* 函数调用 */ }
```

核心：用 `if-else if` 链分发——每个特殊形式对应一个 handler，默认分支是函数调用。

06 答案 — 教学要点总结

五个关键概念

1. 特殊形式 vs 普通调用

普通调用先求值所有参数，特殊形式由 handler 决定求值顺序。

2. 闭包 = 函数 + 环境快照

`env.deepClone()` 保证快照不可变。

3. 参数绑定

通过 `Environment(parent, binds, args)` 三参数构造器实现。

五个关键概念（续）

4. "假"值的扩展

Project 05: 只有 `nil` 为假

Project 06: `nil` 和 `0` 都为假

5. eval 递归深度问题

- 每次 `handleIF` 都递归 `eval` 一次
- 嵌套 `(if ... (if ... (if ...)))` 栈会深
- 连续 `(fn* ...)` 递归调用会栈溢出
- 这就是 Project 07 TCO 要解决的问题!

06 完成 → 具备图灵完备

07 完成 → 解决实际栈溢出

PART 2 · 07-TCO 布置

栈溢出问题 · 尾递归优化 · LFunctionTco · def / defn

为什么需要 TCO?

问题演示

```
(def sum (fn* (n)
  (if (= n 0)
    0
    (+ n (sum (- n 1))))))

(sum 500)
```

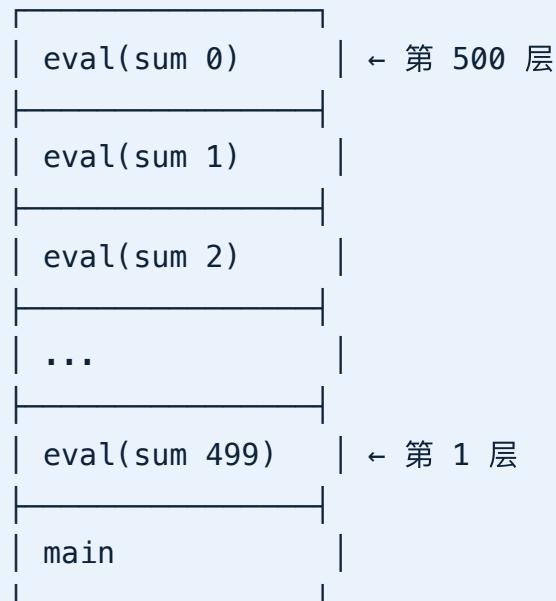
结果: `StackOverflowError` 💣

根本原因

每次 `(sum n)` 调用都会:

1. Java 中 `eval` 递归调用自身
2. JVM 栈帧 +1
3. 递归深度 500 → 500 层栈帧
4. JVM 栈空间默认 512KB → 爆栈

栈帧累积示意



每层都要保留上下文, 因为返回时还需要 `+ n` 运算——无法复用栈帧。

TCO 的核心思想

把递归调用转化为 while 循环——让函数调用不增加栈帧。

递归版 (06-Control)

```
// eval 内
else if (SYMBOL_IF.equals(head))
    return handleIF(list, env);

// handleIF 内部再递归 eval
private static LObject handleIF(
    LList list, Environment env) {
    LObject cond = eval(list.content.get(1), env);
    if (isFalse(cond))
        return eval(list.content.get(3), env);
    return eval(list.content.get(2), env);
}
```

问题：每次 `eval` 都是真递归 → 栈帧增长

TCO 版 (07-TCO)

```
// eval 内 (while 循环)
else if (SYMBOL_IF.equals(head)) {
    obj = handleIF(list, env);
    continue; // 回到循环开头
}

// handleIF 不求值分支, 只返回 AST
private static LObject handleIF(
    LList list, Environment env) {
    LObject cond = eval(list.content.get(1), env);
    if (isFalse(cond)) return list.content.get(3);
    return list.content.get(2);
}
```

关键：handleIF 返回 AST → `while` 循环继续处理。

尾位置 (Tail Position) 的识别

尾位置：求值结果直接作为外层表达式的返回值，不需要再做任何后续处理。

特殊形式	尾位置	非尾位置
(let* bindings body)	body	各个绑定表达式
(do e1 e2 ... en)	en (最后一个)	e1 ~ e_{n-1}
(if cond then else)	then 和 else	cond
(fn* params body)	body (调用时)	—
函数调用 (f a b)	如果 f 是 LFunctionTco，其 body 是尾位置	—

非尾位置示例：(+ n (sum (- n 1))) — (sum ...) 不是尾位置，因为返回后还要做 + n。

TCO 能优化的只有"真·尾调用"——加法后的递归不能优化。

LFunctionTco 类（已提供）

类定义

```
public class LFunctionTco extends LObject {
    public final LList params;    // 参数列表
    public final LObject ast;     // 函数体 AST
    public final Environment env; // 捕获的环境

    public LObject getAst() { return ast; }
    public Environment getEnv() { return env; }

    // 为函数调用创建新环境
    public Environment genEnv(LList args) {
        return new Environment(env, params, args);
    }
}
```

与 LFunction 的区别

特性	LFunction	LFunctionTco
内部实现	Java Lambda	AST + 环境
调用方式	f.call(args)	eval(f.ast, f.genEnv(args))
栈增长	每次调用栈帧+1	循环不增栈
用途	内置函数	用户定义函数

核心：LFunctionTco 把"函数调用"延迟到 eval 的 while 循环中处理，不真正调用 Java 方法。

eval 的 while(true) 循环结构

```
private static LObject eval(LObject obj, Environment env) {
    while (true) {
        if (obj instanceof LSymbol)        return env.lookup((LSymbol) obj);
        else if (obj instanceof LList) {
            LList list = (LList) obj;
            if (list.content.isEmpty())    return obj;
            LObject head = list.content.get(0);
            if (SYMBOL_DEF.equals(head))    { /* 处理 def, return */ }
            else if (SYMBOL_LET.equals(head)) return handleLet(list, env);    // 不 TCO
            else if (SYMBOL_LET_STAR.equals(head)) { obj = handleLetStar(list, env); continue; }
            else if (SYMBOL_DO.equals(head)) { obj = handleDo(list, env); continue; }
            else if (SYMBOL_IF.equals(head)) { obj = handleIF(list, env); continue; }
            else if (SYMBOL_FN_STAR.equals(head)) return handleFnStar(list, env);
            else {
                LObject first = eval(head, env);
                if (first instanceof LFunctionTco) {
                    // TCO 调用: obj = f.getAst(), env = f.genEnv(args), continue
                } else {
                    // 普通 call: return f.call(evaluated)
                }
            }
        } else return obj;
    }
}
```

任务一：TCO 版三个 handler

TODO 1: handleLetStar

```
private static LObject handleLetStar(
    LList list, Environment env) {
    LList bindings = (LList) list.content.get(1);
    for (LObject b : bindings.content) {
        LList binding = (LList) b;
        env.define((LSymbol) binding.content.get(0),
            eval(binding.content.get(1), env));
    }
    return list.content.get(2); // ★ AST
}
```

TODO 2: handleDo

```
private static LObject handleDo(
    LList list, Environment env) {
    int n = list.content.size();
    for (int i = 1; i < n - 1; i++) {
        eval(list.content.get(i), env);
    }
    return list.content.get(n-1); // ★ AST
}
```

TODO 3: handleIF

```
private static LObject handleIF(
    LList list, Environment env) {
    LObject cond = eval(list.content.get(1), env);
    if (cond.equals(new LList())
        || cond.equals(new LInteger(0))) {
        return list.content.get(3); // ★ AST
    }
    return list.content.get(2); // ★ AST
}
```

三者的共同模式

原 06	TCO 版
return eval(body, env)	return body
return eval(branch, env)	return branch

返回 AST 而非求值结果 → 外层 while 循环接管。

任务二：handleFnStar（返回 LFunctionTco）

```
private static LFunctionTco handleFnStar(LList list, Environment env) {
    LList params = (LList) list.content.get(1);
    LObject body = list.content.get(2);
    Environment captured = env.deepClone();
    return new LFunctionTco(params, body, captured);
}
```

和 06-Control 的对比

	06-Control	07-TCO
返回类型	LFunction	LFunctionTco
内部实现	new LFunction(ary, lambda)	new LFunctionTco(params, body, env)
参数绑定	Lambda 内部 new Environment(captured, params, args)	genEnv(args) 方法
调用方式	Lambda 被调用	eval 循环消费 AST

深拷贝环境仍然必需——理由和 06-Control 一样，保证闭包快照不可变。

任务三：eval 的 while 循环核心 (TODO 5)

def 处理 (核心技巧)

```
if (SYMBOL_DEF.equals(head)) {
    LSymbol name = (LSymbol) list.content.get(1);
    LFunctionTco f = handleFnStar(
        (LList) list.content.get(2), GLOBALS);
    env.define(name, f);
    f.getEnv().define(name, f); // ★ 自引用
    return f;
}
```

自引用关键：在函数自身的环境中**再次定义**函数名——这样函数体里可以调用自己（递归）。

LFunctionTco 调用 (TCO 核心)

```
LFunctionTco f = (LFunctionTco) first;
ArrayList<LObject> argsList = new ArrayList<>();
argsList.add(first); // 占位 (index 0)
for (int i = 1; i < list.content.size(); i++) {
    argsList.add(eval(list.content.get(i), env));
}
LList args = new LList(argsList);

obj = f.getAst();           // ★ 替换为函数体
env = f.genEnv(args);       // ★ 切换到新环境
continue;                   // ★ 回循环开头
```

三行核心：改 `obj`、改 `env`、`continue`——**栈帧不增。**

任务四：扩展1 — = 和 < 比较

```
// 在 static 块中添加
GLOBALS.define(new LSymbol("="), new LFunction(2,
  list -> new LInteger(
    ((LInteger) list.content.get(1)).value
    == ((LInteger) list.content.get(2)).value ? 1 : 0)));

GLOBALS.define(new LSymbol("<"), new LFunction(2,
  list -> new LInteger(
    ((LInteger) list.content.get(1)).value
    < ((LInteger) list.content.get(2)).value ? 1 : 0)));
```

约定

- 真 → 返回 `LInteger(1)`
- 假 → 返回 `LInteger(0)`
- 与 `if` 的"假值" (0 或空列表) 无缝衔接

使用示例

```
(if (= n 0) "zero" "non-zero")
(if (< i 100) (recurse) (done))
```

任务五：扩展2 — `defn` 语法糖

语法糖规则

```
(defn name (params) body)
≡ (def name (fn* (params) body))
```

实现 (AST 变换)

```
if (SYMBOL_DEFN.equals(head)) {
    LSymbol name = (LSymbol) list.content.get(1);
    LList params = (LList) list.content.get(2);
    LObject body = list.content.get(3);
    LList fnList = new LList(
        SYMBOL_FN_STAR, params, body);
    LList defList = new LList(
        SYMBOL_DEF, name, fnList);
    obj = defList;
    continue; // 下一轮循环处理 def
}
```

使用对比

```
;; defn (新)
(defn fact (n)
  (if (= n 0) 1
      (* n (fact (- n 1)))))
```

```
;; def + fn* (旧)
(def fact (fn* (n)
  (if (= n 0) 1
      (* n (fact (- n 1)))))
```

Lisp 宏的本质：通过 AST 变换让语法更自然——不改变语义，只改变表达方式。

07-TCO 任务总览与测试

5 个 TODO

TODO	内容	难度
1	<code>handleLetStar</code> 返回 AST	★★
2	<code>handleDo</code> 返回 AST	★
3	<code>handleIF</code> 返回 AST	★
4	<code>handleFnStar</code> 返 LFunctionTco	★★
5	<code>eval</code> 的 while 循环	★★★★★
7	扩展1: <code>=</code> 和 <code><</code>	★
扩展2	<code>defn</code> 语法糖	★★★

核心难点在 TODO 5

- 多个 `continue` 的正确使用
- `def` 的自引用技巧
- LFunctionTco vs LFunction 的分支处理

公开测试 (16 个)

分类	测试数
def 定义	1
TCO 深度 (500/700)	2
控制流回归	5
扩展1: <code>=</code> / <code><</code>	5
扩展2: <code>defn</code>	3

运行测试

```
cd 07-tco
mvn test
```

测试通过验证: `(sum 700)` 不栈溢出 ✓

截止 下次课开始前

TCO 实战验证 — sum 函数

❌ 非尾递归版本（栈溢出）

```
(def sum (fn* (n)
  (if (= n 0)
    0
    (+ n (sum (- n 1))))))

(sum 500) ;; StackOverflow ❌
```

原因：`(+ n (sum ...))` 中 `sum` 的返回值后还要做 `+` ——不是尾调用。

✅ 尾递归版本（TCO 工作）

```
(def sum-iter (fn* (n acc)
  (if (= n 0)
    acc
    (sum-iter (- n 1) (+ acc n))))

(sum-iter 500 0) ;; → 125250 ✓
```

关键：`(sum-iter ...)` 是最后的操作，直接返回——真·尾调用，TCO 可以优化。

提示：用累积器（accumulator）参数把中间结果带过去，就能让递归变成尾递归。

课后作业

必做

1. 完成 Project 07 TCO

提交到 SEECODER-Coder，截止下次课前。

5 个 TODO + 扩展1 + 扩展2

2. AI 辅助实验报告（一段话）

描述你实现 TCO 时遇到的最大挑战（例如 `continue` 的控制流、`def` 的自引用等），记录你是如何用 AI 辅助分析和调试的

选做

- 把 Project 06 的 Fibonacci 改写为尾递归版本，在 07-TCO 上验证能跑 `(fib 1000)`
- 思考：为什么 Java 本身不支持自动 TCO？查阅资料写 200 字说明
- 尝试手动实现一个“循环宏” `(while cond body)` ——用 `def` + 尾递归实现

预习

下周内容：面向对象编程 — 类与对象

带着问题来：Lambda 演算能表达 OOP 吗？闭包和对象的关系？

第八次编程实践课

软件工程与计算 I · 2026 · 南京大学

| 感谢聆听 · 有问题随时提问